

PRODUCT DEMONSTRATION · IN FOUR PARTS

AgentShield

An **Agentic Fraud Detection System for Agentic Payments**.

When an AI agent asks to move money, AgentShield answers first — **ALLOW**, **STEP-UP** or **BLOCK** — at `POST /v1/orders`, before any rupee moves.

PART ONE

The Problem

Agentic payments are shipping right now — and with them a new class of fraud that transaction-fraud models, trained on humans, confidently miss. This is what AgentShield exists to stop.

Prepared by Sagar Khandagre · Passionate About Agentic And Cloud-Native Security

GitHub github.com/sagarkhandagre998

Contents 1 · The problem 2 · The solution 3 · The architecture 4 · Proven live

1 The problem

Traditional fraud asks “is this *transaction* fraudulent?” Agentic payments force a strictly larger question.

A user says “**buy groceries under ₹2,000.**” The agent reads some untrusted tool output and buys a ₹1,800 subscription instead. Every classic signal is clean — known merchant, known customer, known device, valid method — so a transaction-fraud model waves it through. The defect is not in the transaction; it is that **Intent ≠ Transaction**. Closing that gap, before money moves, is the whole product.

THE OLD QUESTION

Is this payment bad?

Probabilistic, learned from human rhythm — session cadence, time on page, the 1 a.m. impulse buy. An agent breaks every one of those signals at once.

THE AGENTIC QUESTION

Can we trust the chain of decisions that produced it?

Was this debit actually asked for — and does this agent’s behaviour look like fraud? A different fraud, needing a different mechanism.

Why now — the rails already shipped

This is not a hypothetical threat surface. In 2025–2026 Visa, Mastercard, Stripe and PayPal all shipped agentic-commerce protocols; Cloudflare’s Agent Pay and the Agent Payments Protocol (AP2) define how an agent acquires a spending mandate and checks out on a human’s behalf. The volume is arriving with them.

\$262_B

AI-AGENT HOLIDAY ORDERS
— SALESFORCE

393%

YOY RISE IN AI-DRIVEN US
RETAIL VISITS — ADOBE

8_B

AGENT REQUESTS IN THE
FIRST TWO MONTHS OF
2026 — DATADOME

~90%

PROJECTED CUT IN THE
COST TO RUN A SCAM,
DRIVING 2× FRAUD — BCG

Seven fraud patterns human rules miss

RISINGWAVE — AGENTIC-PAYMENT FRAUD PATTERNS · AGENTSHIELD IS TESTED AGAINST ALL SEVEN

PATTERN	HOW IT MOVES MONEY
Hijacked Agent Burst	a compromised agent fires many small, in-policy debits that drain a standing mandate faster than the human ever would — the slow drain.
Mandate Replay	a captured mandate or debit is replayed to move the same money twice.
Scope Escalation	a debit steps outside the authorised cap, category or merchant.
Multi-Agent Collusion	one owner splits spend across a fleet of agents so no single view looks abnormal.
Cross-Merchant Correlation	many agents or debits converge on one destination — mule fan-in.
Cadence Anomaly	robotic, too-clean timing that no human session produces.
Geographic Impossibility	a shared device or IP fingerprint reused across agents that cannot be co-located.

Source — RisingWave, *Detecting Fraud in Agentic Payments: 7 Patterns Human Rules Miss*: risingwave.com/blog/detecting-fraud-agentic-payments-7-patterns. The near-1:1 overlap with the families AgentShield is tested against is the strongest external signal that these are the real problems, not invented ones.

2

PART TWO

The Solution

AgentShield — an Agentic Fraud Detection System for Agentic Payments. A deterministic authorization boundary that decides at `POST /v1/orders`, before any rupee moves — with three off-clock ML engines that learn how agents behave and raise the alarm when a debit looks like fraud.

One wire method · three answers · six predicates that can block · three ML engines that can only raise risk

2 The solution

AgentShield answers before the money moves — so a wrong “no” costs a re-confirmation, not a reversal.

When an AI agent asks Razorpay to create an order, the request is shown to AgentShield first. It returns exactly one of three answers — **ALLOW** **STEP-UP** **BLOCK** — and only **ALLOW** proceeds to the payments API and the rails. Budgeted at **p99 ≤ 50 ms**; a live run measures **~3 ms** typical.

TWO INVARIANTS THAT NEVER BEND

- 1 • Only the six predicates (P1–P6) can BLOCK. The ML engines may only raise risk toward a STEP-UP — a probabilistic signal alone never blocks, anywhere in the product.
- 2 • When anything is missing, stale or slow, the system fails closed to a STEP-UP — never an optimistic ALLOW. A blank is never filled with a hopeful zero.

Three off-clock ML engines detect the fraud

EACH LEARNS FROM SETTLED OUTCOMES AND DEPOSITS ONE CALIBRATED FIGURE • NONE CAN BLOCK

ENGINE	AGENTIC FRAUD IT DETECTS	TECHNIQUE
Behavioural-anomaly	a principal acting unlike its own past — amount spikes, velocity bursts, the slow drain, merchant novelty	streaming baselines → gradient-boosted trees + isotonic, floored by an isolation-forest
Graph / GNN	coordinated rings, mule fan-in, synchronised fleets, shared-device collusion	inductive time-aware GraphSAGE above a structural floor + label propagation
Intent-alignment	the debit's merchant / category drifting from the intent sealed once per session	sentence-transformer embeds sealed envelope vs each debit; cosine ⊕ purpose rule

Six deterministic predicates do the enforcing

PREDICATE	FIRES WHEN	VERDICT
P1 replay	idempotency key / presentment id already seen for this token	BLOCK
P2 scope overrun	amount > per-debit cap, or category / merchant outside the overlay	STEP-UP
P3 unbound presentation	no intent envelope, or its digest ≠ the one held	STEP-UP
P4 stale / revoked	token expired, cancellation seen, or block already exhausted	BLOCK
P5 unverifiable identity	caller credential invalid, or token ≠ this customer's	BLOCK
P6 binding mismatch	presented amount ≠ the amount bound to this order at eval time	BLOCK

Otherwise the scorer folds five off-clock figures into a calibrated probability p , multiplies by the rupees at risk, and steps up only when that expected loss exceeds the interruption cost — never a block. The decision function has no **BLOCK** branch at all (`TestDecideNeverBlocks`).

3

PART THREE

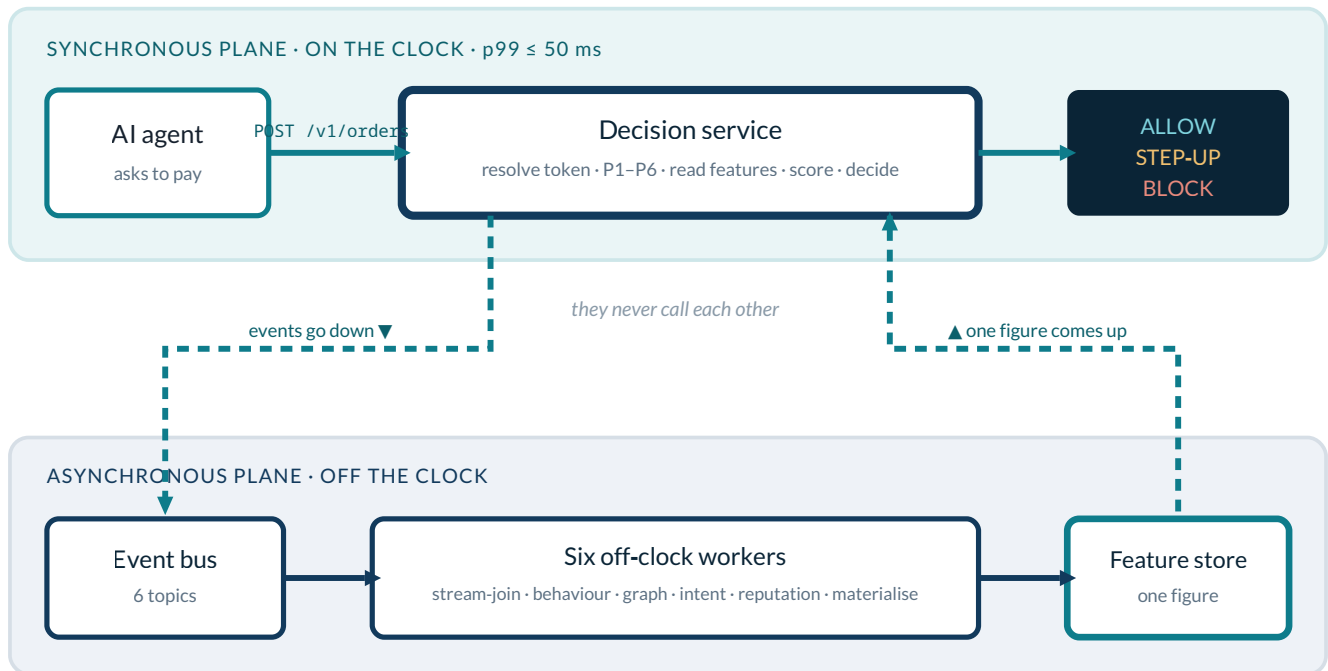
The Architecture

Two planes that never call each other. A stateless on-clock decision service answers in milliseconds; six off-clock workers recompute the baselines, models and reputation. They meet at exactly two points — the event bus (events go down) and the online feature store (one figure comes up).

3 The architecture

Two planes that never call each other — they meet at exactly two points.

A stateless **on-clock** decision service answers in milliseconds; six **off-clock** workers recompute the baselines, models and reputation. Events go *down* to the bus; one calibrated figure comes *up* from the feature store. That is the entire contract between them.



The two planes and their two seams. The on-clock service resolves a token, runs the six predicates, reads one row of precomputed figures, scores and replies — before it publishes anything. Off-clock, six workers fold events into calibrated figures the next request reads by key.

See the full diagrams

SYSTEM DESIGN

Architecture

VISIT →

mermaid.ai/d/eb1955d5-ea21-431e-9e81-057ff2ccd075

DECISION LOGIC

Flow chart

VISIT →

mermaid.ai/d/b07b29a1-97cd-4ba8-9942-f8e7a20cf9b7

END TO END

Sequence diagram

VISIT →

mermaid.ai/d/d89100cd-418b-4450-99e2-08c4ec94f06b

4

PART FOUR

Proven Live

Not a slide claim — a scored run across the full split-process stack. A decision service answering over gRPC, a worker folding events off a real bus, verdicts landing in a durable Postgres chain, then read back and checked against ground truth.

Nine attack families · every verdict correct · on-clock latency far inside budget

4 Proven live

A scored run on the full split-process stack — gRPC in, a real bus, verdicts read back from a durable Postgres ledger.

78/78

DEBITS EVALUATED — 100%
COVERAGE

1.0

DECISION & CODE
ACCURACY VS GROUND
TRUTH

9/9

ATTACK FAMILIES, ALL
PERFECT

0

BROKEN HASH LINKS
ACROSS 78 CHAIN ROWS

The 78 debits split across all three answers exactly as the generator intended — 31 ALLOW / 40 STEP-UP / 7 BLOCK — and the decision the caller saw on the clock is row-for-row the one that landed on the durable, hash-linked chain. Every BLOCK carries a predicate code; the ML engines only ever raised risk to a STEP-UP.

Nine families, every verdict correct

FAMILY	N	DEC	CODE
legit	29	1.0	1.0
velocity_bustout	12	1.0	1.0
shared_device_ring	10	1.0	1.0
mule_fan_in	6	1.0	1.0
synchronised_fleet	6	1.0	1.0
intent_drift	4	1.0	1.0
scope_overrun	4	1.0	1.0
stale_revoked_token	4	1.0	1.0
replay	3	1.0	1.0
total	78	1.0	1.0

On-clock latency

2000 SEQUENTIAL EVALUATE CALLS · BUDGET P99 ≤ 50 MS

STATISTIC	LATENCY
min	1.73 ms
mean	3.16 ms
p50 (median)	3.00 ms
p95	4.53 ms
p99	6.37 ms

Typical ≈3 ms, p99 an order of magnitude under the 50 ms budget — the whole seven-stage clock: gRPC in, three keyed reads, the P1–P6 spine, the feature row, score, decide, reply.

WHAT THE LIVE RUN PROVES

The full on-clock → off-clock loop runs across real transport — gRPC, Redpanda, Redis, Postgres. The six predicates are the sole blockers; fail-closed holds (the 40 step-ups are the guarded middle, not silent allows); and the ledger is tamper-evident, unbroken across all 78 rows. Deterministic seed 7 — the numbers reproduce exactly on a clean slate.